Reviewer Pack — Ehrhart Cohomology Rank Bounds Communication Complexity for …

Entry 68ac6f0e0a3d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 20:23:10 UTC

Ehrhart Cohomology Rank Bounds Communication Complexity for k-CNF

Entry ID: `68ac6f0e0a3d` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-28 20:23:10 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Ehrhart Theory) **Field B** (complexity object): Communication Complexity (k-CNF)

Statement:

> {'main': 'For all n ≥ 3, there exists a constant c > 0 such that for any k-CNF formula with m clauses on n variables, the rank of its Ehrhart cohomology group is at most cmlog(m)', 'counterexample': 'Provide a k-CNF formula with m clauses on n variables whose Ehrhart cohomology group has rank greater than cmln(m).'}

Rationale (proposer's reasoning):

> {'main': 'Ehrhart cohomology provides an algebraic invariant for counting integer points in polytopes, which may reveal structural properties of k-CNF formulas related to their communication complexity.', 'invariant_mapping': 'Construct the incidence vector matrix for the vertices and edges of the polytope defined by the k-CNF formula.'}

Taxonomy category: `AC0_PARITY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `9f4eed21256500fa`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For the k-CNF formula generation, if the average Ehrhart cohomology rank across 30 seeds is less than or equal to c * log(m), where m is the number of clauses and c is a constant, then the conjecture is supported. If any seed produces an Ehrhart cohomology rank greater than c * ln(m), the conjecture is falsified.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"Ehrhart cohomology" AND "communication complexity" AND k-CNF` - `"rank bounds" IN algebraic geometry AND Ehrhart theory AND communication complexity` - `"k-CNF formula" AND Ehrhart cohomology rank AND communication complexity`

Top relevant hits considered: - [http://arxiv.org/abs/1409.1534v1] Algorithms in Real Algebraic Geometry: A Survey - [http://arxiv.org/abs/1611.00827v2] Geometric complexity theory and matrix powering - [http://arxiv.org/abs/1503.08426v1] K3 en route From Geometry to Conformal Field Theory

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 40
    m = random.randint(3, n * (n - 1) // 2)
    clause_density = 0.5

    # Generate a k-CNF formula with m clauses on n variables
    cnf_formula = []
    for _ in range(m):
        num_vars = random.randint(1, n)
        clause = set(random.sample(range(n), num_vars))
        cnf_formula.append(clause)

    # Construct the incidence vector matrix for the polytope defined by the k-CNF formula
    A = [[0] * (2 ** n) for _ in range(m)]
    for i, clause in enumerate(cnf_formula):
        for j in range(1 << n):
            if all((j & (1 << var)) != 0 for var in clause):
                A[i][j] = 1

    # Compute the rank of the incidence vector matrix
    def gaussian_elimination(matrix):
        rows, cols = len(matrix), len(matrix[0])
        rank = 0
        for j in range(cols):
```

```
            i_max = -1
            for i in range(rank, rows):
                if abs(matrix[i][j]) > 1e-9:
                    i_max = i
                    break
            if i_max >= 0:
                matrix[rank], matrix[i_max] = matrix[i_max], matrix[rank]
                for i in range(rows):
                    if i != rank and abs(matrix[i][j]) > 1e-9:
                        factor = -matrix[i][j] / matrix[rank][j]
                        for k in range(cols):
                            matrix[i][k] += factor * matrix[rank][k]
                rank += 1
        return rank

    rank = gaussian_elimination(A)

    # Compute the average Ehrhart cohomology rank over 30 random seeds
    if rank > m * math.log(m):
        return {
            "metric_name": "Ehrhart Cohomology Rank",
            "metric_value": rank,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": f"Formula with {m} clauses and rank {rank}"
        }
    else:
        return {
            "metric_name": "Ehrhart Cohomology Rank",
            "metric_value": rank,
            "instances_tested": 1,
            "conjecture_holds": True,
            "counterexample": ""
        }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) /
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"Formula with {result['metric_value']} clauses a
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_46fd738f.py", line 84, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_46fd738f.py", line 33, in run_trial
    A = [[0] * (2 ** n) for _ in range(m)]
         ~~~~^~~~~~~~~~
MemoryError
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed due to a MemoryError before producing data that could confirm or falsify the conjecture. | next: Increase the memory allocation for the test or optimize the code to handle larger inputs.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11393
2	preregistration	ollama_remote	glm4:latest	0	0	5689
3	novelty	ollama_remote	glm4:latest	0	0	4852
4	novelty	ollama_remote	glm4:latest	0	0	5658
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14314
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9979
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9591
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10039
9	judge	ollama_remote	glm4:latest	0	0	8399

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 79913 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/68ac6f0e0a3d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/68ac6f0e0a3d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/68ac6f0e0a3d.tar.gz` (if generated)